FLARRE AI INNOVATION
CHALLENGE 2026

Proof Before Release

AI-Assisted Verification of Insurance Product Changes

A read-only, obligation-aware verifier for one fictional insurance rule change.

Applicant

Akuthota Pranathi

College

VNR Vignana Jyothi Institute of
Engineering and Technology

Proposal status: planned student prototype.
Synthetic data only; no completed implementation or results claimed.

EXECUTIVE SUMMARY & PROBLEM

The question behind a passing report

Proof Before Release is an AI-assisted verification agent designed to help an insurance product QA reviewer ask: **does each approved requirement have matching, current and correctly scoped test evidence?** AI would propose source-cited testing obligations. Humans would approve their meaning; deterministic code would check the evidence.

The project would test a small, reproducible concept using one fictional deductible change. It would not approve production releases or make underwriting decisions.

Problem statement and target user

Property and casualty (P&C) insurance product rules can describe deductibles, eligibility conditions and effective dates. This prototype models only a minimum deductible, its product and territory, and whether a policy is new or a renewal. The target user is a product QA lead reviewing a configured rule change.

A reviewer may receive a requirement, configuration and green regression report. The report can still refer to an earlier revision, cover the wrong transaction type, omit a required scenario or assert an outdated expected value. The proposal checks these dimensions separately.

Evidence issue	Why it matters in the review
Missing scenario	A passing renewal test does not verify the changed new-policy rule.
Outdated evidence	A recent report timestamp does not establish the tested configuration version.
Mismatched or contradictory values	The approved answer, configured value, test expectation and observed output may disagree.

Business impact: a hypothesis to validate

Earlier identification of these gaps could reduce review rework or reveal an unintended configuration before a human release decision. No carrier incidents, recurring customer demand, financial savings or operational improvements have been established.

Technical basis: GitHub documents that workflow reruns retain the original event's commit SHA/ref [1]. pytest distinguishes skipped and expected-failure outcomes even when a suite remains green [2]. These mechanics support the scenario, not proof of insurance customer demand.

PROPOSED SOLUTION & ILLUSTRATIVE SCENARIO

From clauses to findings

An **obligation** defines what must be verified: source clause, product, territory, transaction type, date scenario, expected behavior and required execution outcome. A business reviewer owns the approved requirement and expected answer; neither AI nor the test's assertion defines its own correctness.

The proposed verification agent

AI proposes: extract structured obligations from short written clauses, cite exact supporting passages and highlight missing or ambiguous scope.
Humans approve: accept or correct each proposal before it becomes an input to verification.
Code verifies: compare declared versions/hashes, scope and dates; check approved/configured/asserted/observed values; distinguish execution outcomes.

> **PASS** means the defined checks have matching evidence under the stated runner assumption. It does **not** guarantee an error-free or safe release. **FAIL** identifies a confirmed mismatch or unmet required scenario. **UNKNOWN** identifies insufficient metadata or unresolved interpretation.

Fictional deductible change

For product P in territory T, the minimum deductible for new policies with inception dates on or after **October 21, 2026** changes from **₹1,000 to ₹2,000**. Earlier new policies and renewals retain ₹1,000. The date boundary is inclusive; other products/territories are unsupported. This is an illustration, not a real customer case or completed experiment.

Illustrative evidence	Finding and corrective action
Only renewal tests pass	FAIL — add executed new-policy scenarios before, on and after the date.
Green report identifies old configuration	FAIL — obtain evidence for the declared current version; timestamp alone is insufficient.
New-policy test expects ₹1,000 after the date	FAIL — reconcile the expectation with the independently approved ₹2,000 answer.
Dataset or configuration identity is missing	UNKNOWN — supply the missing binding before treating evidence as adequate.

A corrected packet would satisfy six declared scenarios: new policies and renewals before, on and after the date. Each evidence-table row would show its source, values, outcome, reason and next action. Humans retain final authority.

SYSTEM DESIGN

Architecture and reviewer workflow

The model assists interpretation; approved obligations and supplied test artifacts feed the deterministic engine. Test artifacts bypass AI analysis. The agent has no independent release authority.



Evidence packet and traceability

Inputs would include requirement text, approved obligations, configuration, test definitions/dataset, a run manifest, JUnit results and a scenario-results sidecar. The sidecar carries scope and expected/observed values that a generic test report may omit.

Each finding would link to its clause and artifact identity. A local record could preserve corrections and review reasons; it would not be an enterprise audit certification. Hashes bind supplied content but do not prove that an honest runner executed it.

IMPLEMENTATION & FEASIBILITY

A bounded prototype, evaluated fairly

The prototype would use synthetic data and one fictional rule family. A practical stack is Python for deterministic logic, Pydantic for typed schemas, a hardened XML parser for JUnit and FastAPI with a simple evidence-table interface. No carrier integrations or broad document-processing platform are required.

Phase	Proposed work
1 • Synthetic inputs	Create the requirement, manually approved obligations, configuration and controlled sample test evidence. Freeze independent expected labels.
2 • Requirement analysis	Add one AI extraction adapter with source citations, ambiguity flags and human correction/approval. Evaluate only if time/access permit.
3 • Deterministic checks	Implement version/hash, product/transaction/date, expected-value and execution-status checks against approved obligations.
4 • Review and evaluation	Build the evidence table; run reproducible fixtures and fair baselines; report errors and limits.

These are proposed phases, not completed components. Manual obligations allow the core verifier to run without the AI stage. A deterministic-only demo would leave AI value proposed; no completed prototype is promised before submission.

Evaluation plan

Separate development cases from 24 held-out synthetic bundles by concrete failure templates, not just different amounts. Fix student-authored expected labels before evaluation, independently of model outputs. Include valid packets and seeded stale, wrong-scope, missing/skipped, contradictory-value and missing-metadata cases.

Baselines: a manual checklist, manual/template obligation extraction and a properly configured deterministic checker with equivalent inputs and checks. For AI ablation, keep the approved obligations and final verifier identical; count correction effort as well as authoring time.

Measures: seeded-gap detection precision/recall, incorrect alerts on valid packets, false-complete classifications, provenance coverage, AI citation/tuple fidelity, abstention and total reviewer effort. A desired outcome is no falsely complete seeded defective packet and no unnecessary blocker on a valid one; these are prospective aims, not results.

Synthetic and self-evaluated cases test specified behavior only. They do not establish carrier demand, population reliability or superiority over an untested Flarre/ALM workflow. AI must show useful interpretation or authoring assistance, not better equality checking.

BENEFITS, SAFEGUARDS & NEXT STEPS

Value to test, limits to preserve

Intended benefits

Earlier visibility of missing or inconsistent evidence; traceability from an approved clause to its test evidence; consistent review criteria; and explainable findings with a corrective action. Reduced review effort is a hypothesis to measure, not proven savings.

Safeguards and limitations

The prototype would use only synthetic inputs and a controlled local runner. Documents are untrusted data: AI gets no execution tools, proposals require schema/source checks and human approval, and uploaded instructions cannot approve findings. XML processing would reject entities/external references, excessive files and unsupported report conventions.

Missing metadata or unresolved meaning stays UNKNOWN. A consistently fabricated unsigned manifest and report could fool the MVP. It cannot certify complete assurance, authentic execution, regulatory compliance or an error-free release. Final release decisions remain with authorized humans; underwriting, policy deployment and legal interpretation are excluded.

Existing solutions and differentiation

Pact Broker already tracks version-specific verification [3]; in-toto verifies declared artifact chains [4]. Properly configured CI or application lifecycle management (ALM) may solve much or all of this task. The contribution to test is an obligation-focused reconciliation view, not new hashing or traceability.

Flarre’s reported requirements, implementation-analysis and testing capabilities create overlap risk. Current product pages were inaccessible in this research; no missing Flarre feature is claimed. Separate-tool need, customer demand and differentiation remain unvalidated. An embedded feature may be the appropriate outcome.

Next validation step

Review one insurance QA packet with its owner and compare an equivalently configured existing workflow. Measure AI authoring benefit before expanding scope. Broader integrations and authenticated runner evidence would follow only if an unmet task remains.

Primary references

[1] GitHub • Re-running workflows and jobs

[2] pytest • How to use skip and xfail

[3] Pact Broker • Can I Deploy

[4] in-toto • README and verification model

Accessed October 12, 2026. Clickable primary documentation supports technical mechanics, not customer demand or achieved results.